



UNIP
UNIVERSIDADE PAULISTA

Roteiros

Programação Aplicada em .NET

Organização do conjunto

Este documento conduz a construção progressiva de uma API .NET voltada ao processamento confiável de pedidos. Cada roteiro termina com verificações, extensões e evidências que alimentam diretamente o relatório correspondente.

As oito atividades usam o domínio didático Hub de Processamento de Pedidos. O professor pode substituir o domínio, desde que preserve os contratos, testes e critérios.

Atividade	Eixo
1	Pattern matching, atributos, reflexão controlada, records e DTOs imutáveis
2	Span<T>, Memory<T>, pooling e caminhos críticos limitados por E/S e medidos
3	Consultas previsíveis, projeções, tracking, consultas compiladas e JSON
4	Channels, SemaphoreSlim, coleções concorrentes e backpressure
5	BenchmarkDotNet, counters, traces, p95/p99 e testes de carga
6	Modularização, IoC, fronteiras e versionamento compatível de APIs
7	SignalR, reconexão, streaming, cache de resposta e ETags
8	WebApplicationFactory, bancos efêmeros, analisadores e revisão automatizada

Verificação: As atividades são cumulativas. Antes de iniciar uma unidade, mantenha uma versão executável da unidade anterior.

Verificação: Antes de cada atividade, criar uma branch ou tag e confirmar que a versão anterior compila, executa e possui configuração reproduzível.



Disciplina: Programação Aplicada em .NET

Título da aula: Pattern matching, atributos, reflexão controlada, records e DTOs imutáveis

ROTEIRO 1

Objetivo

Modelar os contratos e o domínio do Hub de Pedidos com tipos imutáveis, classificação por padrões e descoberta controlada de processadores.

Resultado esperado

Aplicação com contratos imutáveis, classificação testada e catálogo determinístico de processadores.

Etapa 1 – Criar contratos imutáveis

1. Criar a solução HubPedidos com projetos Api, Domain, Application e Tests, todos em net10.0.
2. Modelar PedidoId, ItemPedido, CriarPedidoRequest e PedidoResponse com records ou tipos imutáveis adequados.
3. Impedir quantidades, preços ou identificadores inválidos em uma fronteira bem definida.
4. Criar mapeamentos explícitos entre DTOs e domínio, registrando conversões que podem falhar.

Verificação: Os contratos compilam, preservam invariantes e não compartilham coleções mutáveis inadvertidamente.

Etapa 2 – Classificar com pattern matching

1. Definir categorias de processamento a partir de valor, quantidade de itens, região e prioridade.
2. Implementar a classificação com expressão switch, padrões de propriedade e padrões relacionais/lógicos.
3. Adicionar caso explícito para dados inválidos e revisar a ordem dos braços.
4. Criar testes parametrizados para todas as classes de equivalência identificadas.

Verificação: Cada caso de negócio alcança um braço previsível e os testes documentam os limites.

Etapa 3 – Usar atributos e reflexão com escopo

1. Criar `ProcessadorPedidoAttribute` para associar categoria e versão a uma implementação.
2. Definir uma interface comum para processadores e implementar pelo menos duas estratégias.
3. Inspecionar somente o assembly conhecido, filtrar tipos concretos e validar construtores ou registro no contêiner.
4. Construir o catálogo uma vez na inicialização e falhar de forma legível em duplicidade ou configuração inválida.

Verificação: O catálogo é determinístico, não realiza varredura global e rejeita metadados inconsistentes.

Etapa 4 – Integrar e documentar decisões

1. Criar endpoint que receba o DTO, converta para domínio, classifique e selecione o processador.
2. Retornar resposta tipada sem expor tipos internos do domínio.
3. Comparar registro explícito e descoberta por reflexão para este cenário.

4. Registrar no relatório onde imutabilidade e reflexão melhoraram ou complicaram a solução.

Verificação: O fluxo completo usa contratos estáveis e a escolha de reflexão possui justificativa e limites.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredo, credencial ou dado sensível.
- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

Opcional: escolha uma ou mais extensões para obter a pontuação de aprimoramentos.

- Gerar o catálogo em build com source generator, comparando-o à reflexão.
- Adicionar teste arquitetural para impedir que DTOs dependam da camada de infraestrutura.
- Criar conversões com `TryParse/IParsable<T>` para um value object.

Evidências para o relatório

- Records, value objects, DTOs e mapeadores produzidos para pedidos e itens.
- Expressão switch que classifique pedidos e tabela de casos cobertos.
- Atributo customizado e catálogo de processadores construído com reflexão restrita.
- Testes de igualdade, cópia, validação e descoberta de processadores.

 	Disciplina: Programação Aplicada em .NET Título da aula: Span<T>, Memory<T>, pooling e caminhos críticos limitados por E/S e medidos	ROTEIRO 2
---	--	------------------

Objetivo

Reduzir cópias e alocações no recebimento de lotes de pedidos, comprovando ganhos e custos com benchmarks reproduzíveis.

Resultado esperado

Implementações de referência e otimizada funcionalmente equivalentes, com benchmark reproduzível e gerenciamento seguro de buffers.

Etapa 1 – Medir a implementação inicial

1. Criar importação de pedidos em formato delimitado ou JSON Lines usando APIs simples e corretas.
2. Adicionar testes funcionais antes de qualquer otimização.
3. Criar projeto de benchmark separado e medir tempo e bytes alocados da implementação baseline.
4. Registrar SDK, sistema operacional, processador, tamanho dos dados e configuração do job.

Verificação: Existe uma linha de base reproduzível e funcionalmente validada.

Etapa 2 – Reduzir cópias com spans

1. Identificar uma conversão ou recorte que aloque string/array repetidamente no hot path.
2. Criar versão com `ReadOnlySpan<char>` ou `ReadOnlySpan<byte>` para parsing síncrono.

3. Evitar armazenar spans além do escopo permitido ou atravessar await.
4. Reexecutar os testes de equivalência e o benchmark.

Verificação: A versão otimizada mantém o contrato e a diferença de alocação é mensurável.

Etapa 3 – Aplicar pooling com segurança

1. Escolher o mecanismo conforme o contrato: `ArrayPool<byte>.Shared.Rent` para arrays ou `MemoryPool<byte>.Shared.Rent` para obter um `IMemoryOwner<byte>`.
2. No `ArrayPool`, devolver o array com `Return` em `finally`; no `MemoryPool`, liberar o proprietário com `Dispose` ou `using`, inclusive em falha e cancelamento.
3. Processar somente a fatia válida e não assumir que o buffer alugado está zerado.
4. Limpar a região utilizada quando o cenário simular dado sensível e justificar o custo.

Verificação: O ciclo de vida do buffer é demonstrável e não há uso após devolução.

Etapa 4 – Otimizar o fluxo assíncrono

1. Ler o stream incrementalmente com limite total e `CancellationToken`.
2. Evitar flush por item; agrupar escrita quando o contrato permitir.
3. Executar teste de lote pequeno e grande, registrando throughput e memória.
4. Aceitar, simplificar ou reverter a otimização conforme os resultados.

Verificação: A decisão final compara ganho, variabilidade, memória e custo de manutenção.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredo, credencial ou dado sensível.
- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

Opcional: escolha uma ou mais extensões para obter a pontuação de aprimoramentos.

- Adicionar `MemoryPool<T>` com propriedade explícita do buffer.
- Comparar parsing UTF-8 direto com desserialização intermediária.
- Criar um limite adaptativo de lote e medir seu impacto.

Evidências para o relatório

- Implementações baseline e otimizada do parser ou normalizador de lote.
- BenchmarkDotNet com tempo, alocações, configuração, versão e ambiente registrados.
- Código de aluguel/devolução de buffer e demonstração do caminho de erro/cancelamento.
- Tabela de decisão que aceite ou rejeite a otimização com base em evidência e complexidade.

Objetivo

Persistir e consultar pedidos com EF Core, analisando SQL, roundtrips, tracking e impacto dos mapeamentos adotados.

Resultado esperado

Modelo persistido e consultas paginadas previsíveis, com SQL, tracking e mapeamentos analisados e testados.

Etapa 1 – Modelar e observar o SQL

1. Mapear Pedido, ItemPedido e um tipo complexo de endereço ou metadados.
2. Criar migration, revisar nomes, chaves, índices e nulabilidade antes de aplicar.
3. Habilitar logging de comandos sem valores sensíveis e capturar o SQL de uma consulta simples.
4. Preparar massa de dados suficiente para tornar paginação e agregações observáveis.

Verificação: O esquema é reproduzível e o SQL pode ser associado à consulta LINQ correspondente.

Etapa 2 – Construir consultas enxutas

1. Criar DTO de resumo e projetar diretamente no banco.
2. Aplicar filtros, ordenação determinística, paginação e contagem total.
3. Quando a entidade for alterada pelo mesmo DbContext, preferir consulta com tracking; se usar no-tracking, documentar e testar a anexação ou atualização explícita.

4. Comparar carregamento relacionado, projeção e consulta dividida quando aplicável.

Verificação: O painel evita N+1 e materializa somente os dados exigidos pelo contrato.

Etapa 3 – Avaliar consulta compilada

1. Selecionar uma consulta estável e repetida, com parâmetros escalares.
2. Criar versão com EF.CompileAsyncQuery compatível com um único modelo.
3. Medir as duas versões separando tempo de banco e overhead do cliente quando possível.
4. Manter a versão simples se o ganho não justificar a complexidade.

Verificação: A adoção ou rejeição da consulta compilada é sustentada por medição e restrições conhecidas.

Etapa 4 – Comparar mapeamento relacional e JSON

1. Verificar a documentação do provedor e da versão antes de habilitar JSON.
2. Mapear um agregado pequeno como tipo complexo/JSON quando suportado, mantendo alternativa relacional.
3. Executar filtro e atualização e inspecionar o SQL produzido.
4. Registrar impacto em tracking, portabilidade, índice e evolução do esquema.

Verificação: A decisão de mapeamento considera consultas reais, não apenas conveniência do modelo.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredo, credencial ou dado sensível.

- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

Opcional: escolha uma ou mais extensões para obter a pontuação de aprimoramentos.

- Comparar paginação por deslocamento e por cursor.
- Medir consulta dividida versus consulta única em um grafo controlado.
- Adicionar teste que falhe ao detectar quantidade excessiva de comandos SQL.

Evidências para o relatório

- Modelo, configurações e migration de pedidos, itens e metadados.
- Consultas de painel com projeção, paginação e AsNoTracking, acompanhadas do SQL gerado.
- Comparação entre consulta normal e compilada para um caso repetitivo.
- Análise do mapeamento relacional versus JSON e suas limitações no provedor escolhido.

 	Disciplina: Programação Aplicada em .NET Título da aula: Channels, SemaphoreSlim, coleções concorrentes e backpressure	ROTEIRO 4
---	--	------------------

Objetivo

Processar pedidos por uma fila limitada, controlando produtores, consumidores, acesso ao banco e encerramento da aplicação.

Resultado esperado

Fila limitada com produtores e consumidores coordenados, limite de concorrência e política de fila cheia comprovados por métricas.

Etapa 1 – Criar um canal limitado

1. Definir ProcessarPedidoJob imutável com identificador e dados mínimos.
2. Registrar um Channel<ProcessarPedidoJob> bounded como serviço único.
3. Escolher a capacidade e um FullMode (Wait, DropNewest, DropOldest ou DropWrite), documentando espera ou descarte.
4. Criar endpoint produtor que respeite cancelamento; se houver rejeição, implementá-la explicitamente com TryWrite e resposta HTTP definida, sem manter contexto EF no item.

Verificação: A fila tem capacidade observável e comportamento definido quando cheia.

Etapa 2 – Implementar consumidores

1. Criar BackgroundService que leia com ReadAllAsync e token de parada.
2. Criar escopo de DI por item ou lote e obter dependências dentro desse escopo.
3. Registrar sucesso, falha e duração sem perder o identificador do pedido.

4. Concluir ou drenar o canal conforme a política de encerramento escolhida.

Verificação: Os consumidores encerram de forma coordenada e falhas ficam visíveis.

Etapas 3 – Limitar banco e compartilhar estado

1. Aplicar SemaphoreSlim ao trecho que usa a dependência limitada.
2. Liberar o semáforo em finally e evitar segurar o slot durante trabalho não relacionado.
3. Usar ConcurrentDictionary para métricas de estado somente com operações atômicas apropriadas.
4. Relacionar limite do semáforo ao pool de conexões e à carga de outras rotas.

Verificação: O teste confirma o teto de concorrência e não há slot perdido em exceções.

Etapas 4 – Demonstrar backpressure

1. Produzir carga acima da capacidade com taxa controlada.
2. Registrar profundidade da fila, tempo de espera e itens aceitos, processados, descartados e rejeitados, distinguindo as duas últimas categorias.
3. Comparar ao menos duas combinações de capacidade e consumidores.
4. Escolher configuração didática e explicar por que aumentar paralelismo pode piorar o banco.

Verificação: A política estabiliza uso de recursos e seus custos aparecem nas métricas.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredo, credencial ou dado sensível.

- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

Opcional: escolha uma ou mais extensões para obter a pontuação de aprimoramentos.

- Criar filas separadas por prioridade com prevenção de starvation.
- Adicionar idempotência persistida para reprocessamentos.
- Implementar processamento em lote e comparar throughput/latência.

Evidências para o relatório

- Configuração do canal limitado e política escolhida quando a fila está cheia.
- Worker com consumidores, cancelamento, tratamento de falhas e conclusão coordenada.
- Limite de concorrência para persistência e justificativa em relação ao pool de conexões.
- Gráficos ou tabela de fila, throughput, descartes, rejeições e latência sob duas configurações.

 	Disciplina: Programação Aplicada em .NET Título da aula: BenchmarkDotNet, counters, traces, p95/p99 e testes de carga	ROTEIRO 5
---	---	------------------

Objetivo

Diagnosticar o Hub sob carga, estabelecer metas de latência e investigar gargalos com evidências reproduzíveis.

Resultado esperado

Diagnóstico reproduzível com carga, contadores, trace e comparação antes e depois de uma intervenção.

Etapa 1 – Definir hipótese e meta

1. Escolher uma rota de criação, uma rota de consulta e um método crítico isolável para BenchmarkDotNet.
2. Definir hipótese de gargalo e metas didáticas de p95, p99, throughput e taxa de erro.
3. Registrar hardware, SO, SDK, banco, massa de dados e configuração da fila.
4. Preparar um projeto de benchmark e um script de carga versionados, com taxa ou concorrência explícita.

Verificação: O experimento pode ser repetido e possui critério de sucesso antes da execução.

Etapa 2 – Observar counters

1. Executar carga com aquecimento e iniciar dotnet-counters monitor ou collect.
2. Observar CPU, heap, GC, alocação, exceções e ThreadPool, além de métricas do canal.
3. Correlacionar mudanças de fila e latência com os sinais de processo.

4. Salvar a coleta ou transcrever os valores relevantes com horário.

Verificação: Os counters ajudam a escolher o próximo diagnóstico, sem concluir causa isoladamente.

Etapas 3 – Coletar e analisar trace

1. Coletar um trace curto durante a condição problemática.
2. Converter ou abrir o arquivo em ferramenta compatível e localizar CPU, espera ou starvation.
3. Relacionar stacks e eventos ao código e à hipótese inicial.
4. Descartar explicações não sustentadas pelos dados.

Verificação: Existe uma cadeia de evidências entre sintoma, trace e trecho investigado.

Etapas 4 – Intervir e comparar

1. Aplicar uma mudança pequena: consulta, alocação, limite ou batching.
2. Repetir a carga no mesmo ambiente e calcular p50/p95/p99, throughput e erros.
3. Executar os testes funcionais e verificar novo consumo de recursos.
4. Concluir se a hipótese foi confirmada, refutada ou permaneceu inconclusiva.

Verificação: A comparação antes/depois informa a decisão e explicita incertezas.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredo, credencial ou dado sensível.
- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

Opcional: escolha uma ou mais extensões para obter a pontuação de aprimoramentos.

- Adicionar cenário de carga em degraus para localizar saturação.
- Comparar trace com Visual Studio Profiler no mesmo problema.
- Criar orçamento de performance executado periodicamente em ambiente controlado.

Evidências para o relatório

- Plano de medição com hipótese, métricas, carga, ambiente e critérios de sucesso.
- Relatório de BenchmarkDotNet do método selecionado e captura de counters durante o teste de carga.
- Trace analisado com evidência do gargalo ou ausência dele.
- Tabela p50/p95/p99, throughput, erros e conclusão antes/depois de uma intervenção.

 	Disciplina: Programação Aplicada em .NET Título da aula: Modularização, IoC, fronteiras e versionamento compatível de APIs	ROTEIRO 6
---	--	------------------

Objetivo

Separar responsabilidades do Hub e evoluir o contrato HTTP sem romper um cliente que depende da versão anterior.

Resultado esperado

Aplicação modularizada com dependências verificáveis, IoC consistente e versões v1 e v2 compatíveis com o cliente anterior.

Etapa 1 – Explicitar módulos e dependências

1. Definir módulos de pedidos, processamento e notificações com responsabilidades claras.
2. Mover interfaces para a fronteira que as consome e manter implementações na infraestrutura.
3. Criar testes de referência ou inspeção para impedir dependências proibidas.
4. Registrar decisões arquiteturais curtas, incluindo alternativas descartadas.

Verificação: A direção das dependências é verificável no projeto, não apenas no diagrama.

Etapa 2 – Organizar IoC

1. Centralizar registros em extensões por módulo chamadas pelo composition root.
2. Revisar singleton, scoped e transient conforme estado e recursos usados.
3. Evitar resolver serviços manualmente dentro da regra de negócio.
4. Criar teste de inicialização que valide o grafo principal de dependências.

Verificação: O contêiner constrói os fluxos e não contém dependência cativa conhecida.

Etapa 3 – Definir a política de versão

1. Selecionar o endpoint de consulta de pedido e listar consumidores conhecidos.
2. Definir forma de leitura da versão, versão padrão e resposta para versão não suportada.
3. Implementar v1 sem alteração e v2 com uma mudança incompatível deliberada e documentada.
4. Gerar documentação/OpenAPI separável por versão quando o mecanismo adotado suportar.

Verificação: A política é previsível e as duas versões podem ser testadas isoladamente.

Etapa 4 – Provar compatibilidade

1. Criar cliente ou fixture que represente o front-end v1.
2. Executar o cliente contra a aplicação que também contém v2.
3. Adicionar headers de depreciação ou documentação de ciclo de vida quando aplicável.
4. Definir critérios para observar uso e remover uma versão no futuro.

Verificação: A evolução não rompe o consumidor v1 e existe política de transição documentada.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredo, credencial ou dado sensível.
- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

Opcional: escolha uma ou mais extensões para obter a pontuação de aprimoramentos.

- Adicionar teste arquitetural automatizado entre assemblies.
- Criar adaptador anticorrupção entre duas representações de pedido.
- Gerar cliente para v1 e v2 e comparar diferenças de compilação.

Evidências para o relatório

- Diagrama simples de módulos e direção das dependências.
- Composition root e registros de DI com tempos de vida justificados.
- Versões v1 e v2 do endpoint selecionado e matriz de mudanças.
- Teste de compatibilidade provando que o cliente v1 continua funcional.

 	Disciplina: Programação Aplicada em .NET Título da aula: SignalR, reconexão, streaming, cache de resposta e ETags	ROTEIRO 7
---	---	------------------

Objetivo

Notificar o cliente sobre o processamento de pedidos e reduzir transferências de leitura com cache e validação condicional corretos.

Resultado esperado

Cliente autorizado notificado por SignalR, com reconexão, reconciliação HTTP, ETag e cache seguro testados.

Etapa 1 – Publicar eventos com SignalR

1. Criar PedidosHub e contrato do evento PedidoAtualizado.
2. Autorizar no servidor antes de adicionar o cliente ao grupo por pedido ou usuário; não tratar o grupo como mecanismo de segurança.
3. Publicar evento depois da mudança persistida, incluindo versão/instante e dados mínimos.
4. Evitar enviar o objeto de domínio ou dados sensíveis diretamente pelo hub.

Verificação: O evento tipado chega somente aos clientes autorizados e pode ser correlacionado ao pedido.

Etapa 2 – Tratar reconexão e reconciliação

1. Configurar cliente com reconexão automática e callbacks de estado.
2. Desabilitar ações dependentes da conexão e informar reconexão ao usuário.

3. Revalidar a autorização e reingressar somente nos grupos permitidos após nova conexão; remover associações quando a permissão for revogada.
4. Consultar o estado atual via HTTP após reconectar para cobrir eventos perdidos.

Verificação: A interface recupera estado correto após interrupção sem depender de replay implícito.

Etapa 3 – Implementar ETag

1. Definir quais campos compõem a representação e gerar ETag estável para ela.
2. Incluir ETag na resposta de consulta de pedido.
3. Avaliar If-None-Match e devolver 304 quando a representação não mudou.
4. Adicionar testes para mudança, estabilidade e valor inválido/obsoleto.

Verificação: As respostas condicionais seguem a semântica HTTP e reduzem corpo transferido.

Etapa 4 – Aplicar cache com limites

1. Selecionar uma leitura pública e idempotente ou definir política segura que exclua ou varie respostas autenticadas e personalizadas.
2. Registrar UseOutputCache após UseAuthentication e UseAuthorization e configurar duração e variação por rota e query relevantes.
3. Invalidar por tag ou mecanismo equivalente após mudança do pedido.
4. Medir acertos, latência e risco de inconsistência em teste controlado.

Verificação: O cache melhora a leitura prevista sem misturar usuários ou servir versão incompatível.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredo, credencial ou dado sensível.
- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

Opcional: escolha uma ou mais extensões para obter a pontuação de aprimoramentos.

- Implementar streaming de progresso com cancelamento.
- Usar If-Match para atualização otimista e tratar 412.
- Adicionar teste de reconexão com proxy de falha controlado.

Evidências para o relatório

- Hub, grupos e evento tipado de mudança de status.
- Cliente com estados conectado, reconectando e desconectado e estratégia de reconciliação.
- Resposta com ETag e exemplos de GET normal, If-None-Match e 304.
- Política de output caching, chaves de variação, invalidação e rotas excluídas.

 	Disciplina: Programação Aplicada em .NET Título da aula: WebApplicationFactory, bancos efêmeros, analisadores e revisão automatizada	ROTEIRO 8
---	--	------------------

Objetivo

Criar uma rede de segurança executável para contratos, persistência, concorrência e estilo, integrada ao fluxo de revisão.

Resultado esperado

Suíte de integração isolada e determinística, executada por revisão automatizada com formatação e analisadores.

Etapa 1 – Criar a fábrica de testes

1. Adicionar projeto de integração e WebApplicationFactory<Program>.
2. Usar WithWebHostBuilder ou fábrica customizada para substituir serviços externos.
3. Configurar ambiente de teste e impedir uso acidental do banco de desenvolvimento.
4. Criar cliente HTTP com comportamento de redirecionamento e autenticação definido.

Verificação: A aplicação inicia em memória com configuração isolada e dependências controladas.

Etapa 2 – Usar banco efêmero

1. Adotar SQLite em memória para o núcleo do exercício ou contêiner efêmero aprovado.
2. Manter a conexão SQLite aberta durante o teste quando esse provedor for usado.
3. Criar esquema/migrations e dados mínimos por fixture.
4. Documentar diferenças do provedor produtivo que ainda exigem testes específicos.

Verificação: Os testes são isolados, repetíveis e não confundem fidelidade limitada com equivalência total.

Etapa 3 – Cobrir contratos críticos

1. Testar criação e consulta de pedido com persistência real do ambiente de teste.
2. Testar v1/v2, ETag/304 e uma autorização ou validação relevante.
3. Testar produtor da fila com um consumidor/fake observável sem usar atraso arbitrário longo.
4. Executar a suíte repetidamente e em ordem aleatória quando a ferramenta permitir.

Verificação: Os contratos mais arriscados possuem testes determinísticos e mensagens úteis.

Etapa 4 – Automatizar análise e revisão

1. Criar .editorconfig com regras de estilo e qualidade selecionadas.
2. Habilitar análise no build e elevar a erro apenas regras aceitas pela equipe.
3. Executar restore, build, dotnet format --verify-no-changes, testes e coleta de resultados em pipeline ou script.
4. Provocar uma falha de teste e uma de analisador para confirmar o bloqueio da revisão automatizada.

Verificação: A revisão automatizada falha cedo, explica a causa e é reproduzível localmente.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredo, credencial ou dado sensível.

- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

Opcional: escolha uma ou mais extensões para obter a pontuação de aprimoramentos.

- Adicionar teste de contrato por snapshot com revisão explícita de mudanças.
- Executar testes de integração em paralelo após eliminar estado compartilhado.
- Publicar cobertura e resultados de teste como artefatos, sem usar cobertura isoladamente como meta de qualidade.

Evidências para o relatório

- Fábrica customizada, banco efêmero e fixtures de integração.
- Testes HTTP de criação, consulta, versão, ETag e fluxo assíncrono selecionado.
- .editorconfig e configuração de analisadores com justificativa das severidades.
- Workflow ou script de revisão que falhe em teste/análise e produza resultados legíveis.